1. 개요

1.1 작성 목적 및 배경

본 보고서는 오픈소스 자동화 서버인 Jenkins에서 발견된 중대한 취약점인 CVE-2024-23897의 기술적 메커니즘을 분석하고, 이를 통해 발생할 수 있는 보안 위협을 식별하여 효과적인 대응 방안을 제시하는 데 목적이 있습니다.

Jenkins는 오픈소스 자동화 서버(*CI/CD 서버)로 사용자가 저장소에 올린 코드를 보고 빌드, 실행, 배포까지를 수행함

*CI/CD 서버

- CI(Continuous Integration): 개발자가 작성한 코드를 빌드 및 실행하는 과정
  - 저장소에 코드가 변경되면 코드를 읽어보기
  - 빌드 및 테스트 실행
  - 실행 결과에 따른 검증
- CD(Continuous Delivery/Deployment): 빌드된 소프트웨어를 배포하는 과정
  - 배포 직전까지는 자동화
  - 실제 배포는 사람이함

1.2 분석 대상 및 범위

- **대상**: Jenkins CI/CD 서버 (LTS 및 주간 릴리즈 버전)
- **범위**: Jenkins CLI(Command Line Interface) 내의 expand-at 기능에 의한 임의 파일 읽기(Arbitrary File Read) 취약점 및 이와 연계된 원격 코드 실행(RCE) 가능성 분석.

1.3 요약

CVE-2024-23897은 Jenkins CLI의 인자 파싱 과정에서 사용하는 `args4j` 라이브러리의 기능 오용으로 인해 발생합니다. 공격자는 특정 문자를 이용해 서버 내부의 임의 파일을 읽을 수 있습니다.

---

2. 취약점 정보

2.1 CVE 기본 정보

- CVE ID: CVE-2024-23897
- 취약점 유형: Arbitrary File Read (임의 파일 읽기)
- CVSS Score: 9.8 (Critical)

- 공개 날짜: 2024년 1월 24일
- CWE: CWE-22 (Path Traversal)

2.2 영향받는 소프트웨어 및 버전

- Jenkins Weekly: 2.441 미만 버전
- Jenkins LTS: 2.426.3 미만 버전

2.3 주통기(주요정보통신기반시설) 진단 항목 매핑

- **서비스 관리**: U-22 (Log 파일 및 중요 설정 파일 접근 제한 미흡)
- **애플리케이션 보안**: 소프트웨어 개발 보안 가이드 내 '부적절한 입력 확인 및 검증' 항목에 해당

---

3. 취약점 상세 분석

3.1 취약점 발생 원리

Jenkins CLI는 명령 인자를 처리할 때 자바의 args4j 라이브러리를 사용합니다.
이 라이브러리에는 @ 문자 뒤에 파일 경로가 오면 해당 파일의 내용을 읽어 인자로 치환하는 `expand-at` 기능이 기본적으로 활성화되어 있습니다.
Jenkins 서버는 CLI 요청을 받을 때 이 기능을 제한하지 않아, 공격자가 CLI 명령의 인자로 @/etc/passwd 등을 전송하면 서버가 해당 파일을 읽어 처리하게 됩니다.

아래는 args4j의 코드 중 `expand-at` 기능의 일부를 단순화한 것

```
...
private String[] expandAtFiles(String[] args) throws IOException {
    List<String> result = new ArrayList<>();

    for (String arg : args) {
        if (arg.startsWith("@")) {
            File file = new File(arg.substring(1));
            result.addAll(readAllLines(file));
        } else {
            result.add(arg);
        }
    }
    return result;
}
...
```

1. args에 입력값을 받음
2. for문으로 돌면서 @로 시작하는지 검사
3. @로 시작하면 @뒤부터 파일로 판단해서 내용을 읽은 다음에 result에 저장

4. 저장한 파일들을 반환

3.2 주요 트리거 조건

- **CLI 접근 가능성**: Jenkins 서버의 CLI 인터페이스가 활성화되어 있어야 합니다 (기본값 활성)
- Jenkins CLI를 가지고 있을 것

3.2 주요 트리거 조건

- **공격 벡터**: HTTP 또는 SSH를 통한 Jenkins CLI 프로토콜 활용.
- **페이로드 예시**:

```
# Jenkins CLI 클라이언트를 이용한 파일 읽기 시도
java -jar jenkins-cli.jar -s (jenkins 페이지) help "@(읽고 싶은 파일의 경로)"
java -jar jenkins-cli.jar -s (jenkins 페이지) get job "@(읽고 싶은 파일의 경로)"
```

`help` 명령어는 인자를 하나만 받지만, 파일 내용이 여러 줄일 경우 Jenkins는 "해당 인자를 이해할 수 없다"는 오류 메시지와 함께 파일의 내용을 화면에 출력합니다.
get job 명령어의 경우는 존재하지 않는 job 이름이 전달되면 에러로 입력값을 반환합니다.

---

4. 실습 환경

4.1 인프라 구성도

- Dockerfile

```
FROM jenkins/jenkins:2.441

ENV JAVA_OPTS="-Djenkins.install.runSetupWizard=false"

USER root

RUN mkdir -p /usr/share/jenkins/ref/init.groovy.d

COPY set-unsecured-auth.groovy
/usr/share/jenkins/ref/init.groovy.d/setunsecured-auth.groovy

RUN mkdir -p /usr/share/jenkins/ref/secrets \ && echo
'FLAG{8Y3_8Y3_C7F..H3LL0_CV3!!}' > /usr/share/jenkins/ref/secrets/flag \ &&
chown -R jenkins:jenkins /usr/share/jenkins/ref/secrets

USER jenkins
```

- set-unsecured-auth.groovy

```
import jenkins.model.*

import hudson.security.*

def instance = Jenkins.get()

// Disable all security (Anyone can do anything)
instance.setAuthorizationStrategy(new AuthorizationStrategy.Unsecured())
instance.setSecurityRealm(new HudsonPrivateSecurityRealm(false))

instance.save()
```

4.2 타켓 시스템 스펙 및 설정

- **서버**: Jenkins 2.441 서버

4.3 공격자 환경 및 도구

- **운영체제**: Windows11
- **사용 도구**: Powershell

---

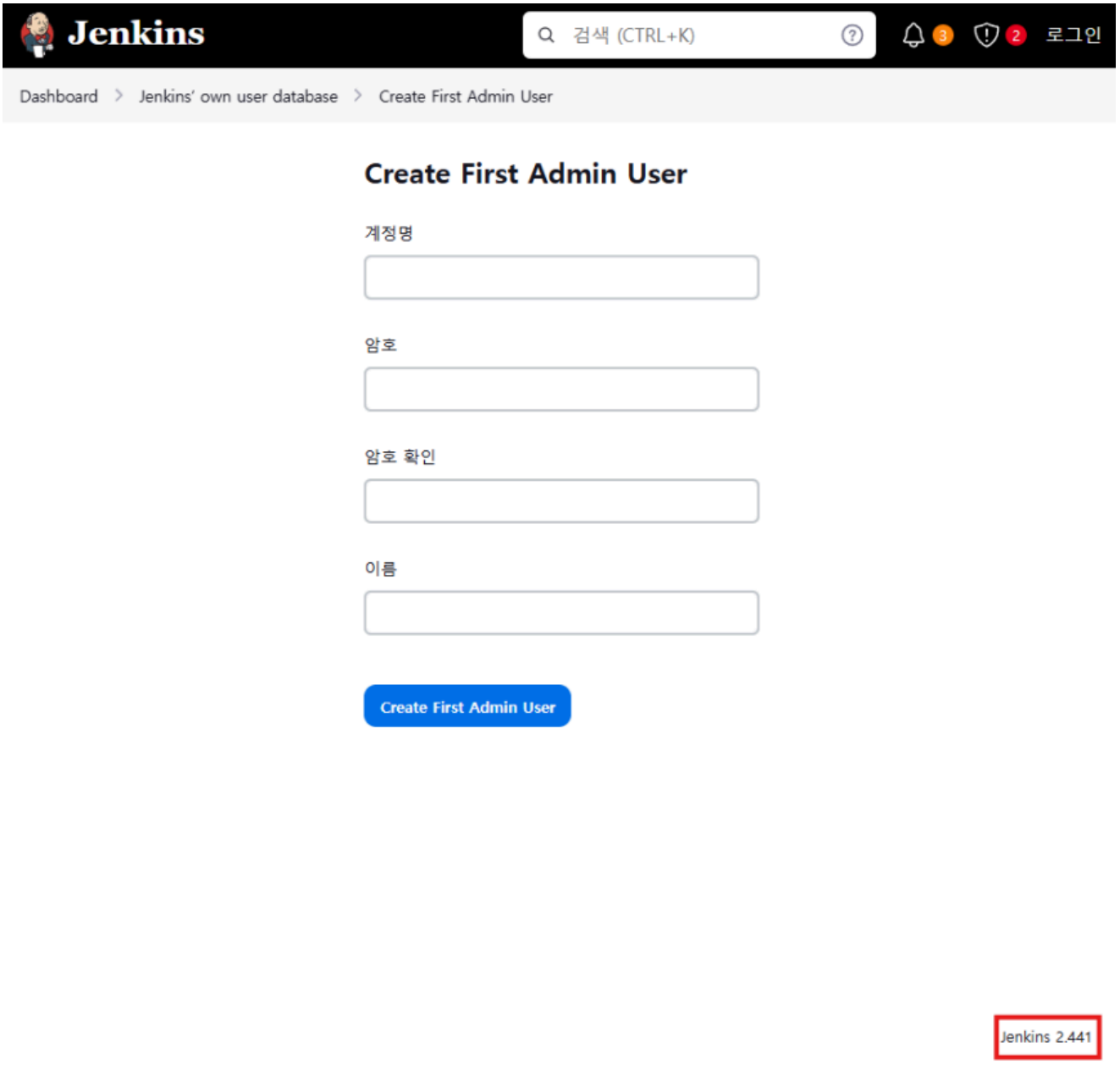
5. 공격 재현 과정(POC)

5.1 공격 시나리오

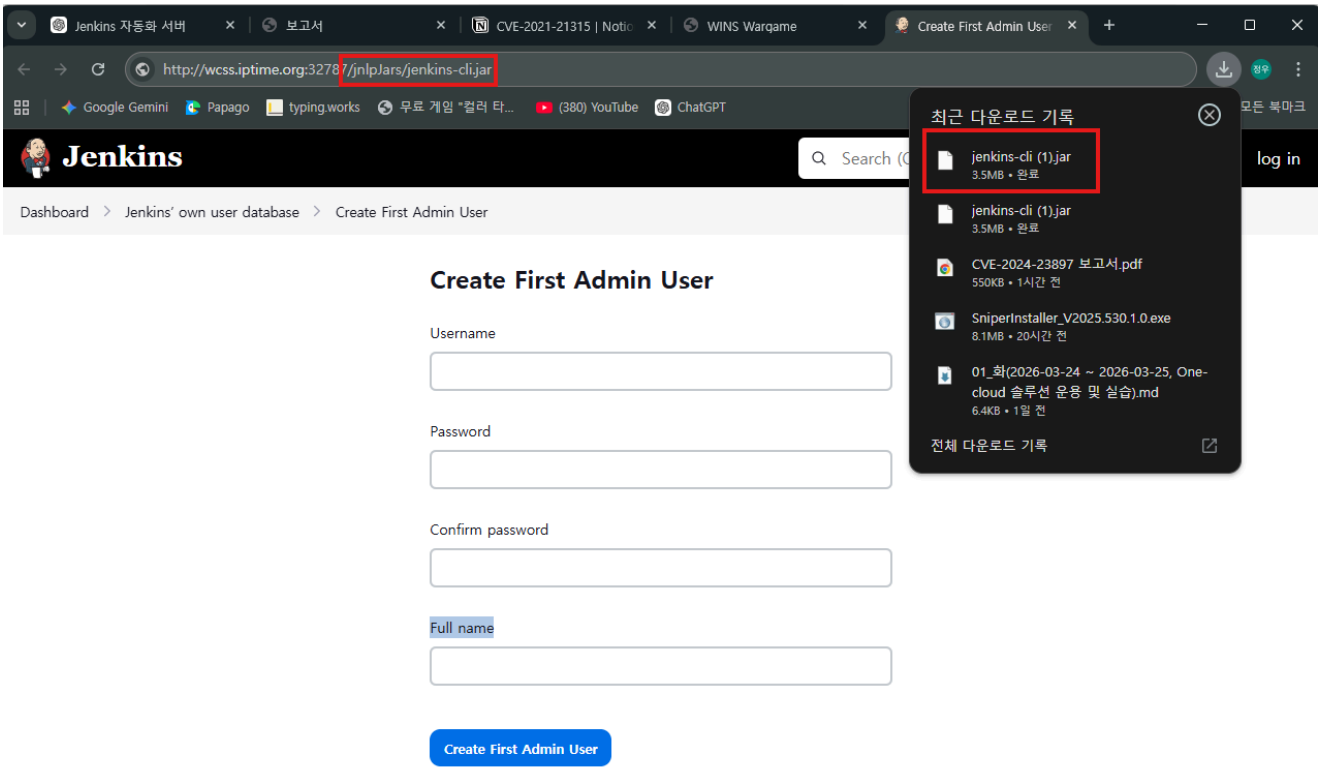
공격자는 Jenkins 서버의 버전이 2.441 이하인 것을 확인하면 곧바로 Jenkins_cli를 다운받아서 취약점을 공격할 수 있다.

5.2 익스플로잇 수행 과정

Jenkins 서버의 버전이 2.441 이하이므로 CVE-2024-23897 취약점이 성립할 수도 있음



jenkins-cli.jar이 필요하니 서버주소 뒤에 /jnlpJars/jenkins-cli.jar 를 붙여서 가져온다.

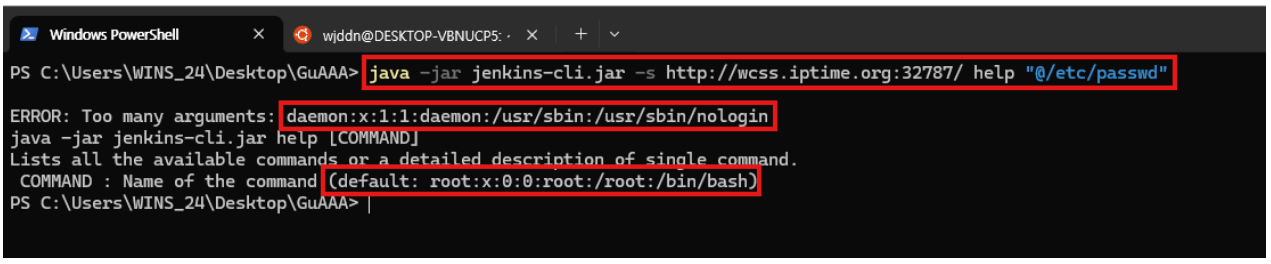


/etc/passwd의 값을 읽어오기 위해서 `java -jar jenkins-cli.jar -s http://wcss.iptime.org:33134/ (get-job | help) "@/etc/passwd"` 를 통해서 /etc/passwd를 읽어온다.

위 명령어를 좀 더 자세히 보면

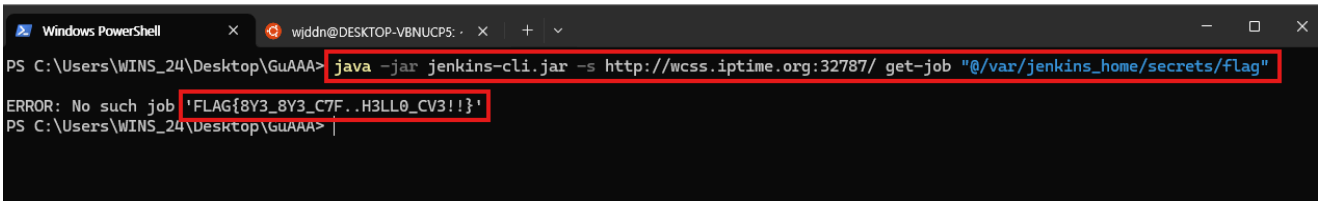
- `java -jar jenkins-cli.jar` : jar파일인 jenkins-cli.jar를 실행. jenkins-cli를 통해서 서버를 통하지 않고 원격으로 명령을 보낼 수 있음
- `-s http://wcss.iptime.org:33134/` : 명령을 보낼 대상인 Jenkins 서버 주소
- (get-job | help)
  - get-job : Jenkins CLI에서 제공하는 명령어로 작업의 설정 정보를 가져옴. Linux의 명령어와는 다른 Jenkins CLI의 명령어임
  - help : get-job 과 같이 Jenkins CLI에서 제공하는 명령어로 뒤에 들어온 인자에 관한 도움말을 반환함. 마찬가지로 리눅스 명령어가 아닌 Jenkins CLI의 명령어
- "@/etc/passwd" : 이 부분이 핵심으로 Jenkins CLI에서는 @file 을 하게 되면 해당 파일의 내용을 반환하는데 이때 입력값의 검증과정이 없어서 읽을 수 없는 파일도 읽어서 반환 할 수 있게 된다.
- `get-job` 으로 읽은 경우

- `help`로 읽은 경우



특이사항으로는 `get-job`과 `help` 둘 다 /etc/passwd의 전체 값이 아니라 첫 번째 줄 또는 첫 번째 줄과 두 번째 줄만 보이는 등 일부만 보인다. /etc/passwd의 파일 내용이 여러 줄로 되어있어 인자를 여러 개를 주는데 get-job과 help는 인자를 하나만 받기 때문에 입력된 값들 중 일부만 반환하게 된다.

플래그값의 위치는 /var/jenkins_home/secrets/flag 이므로 해당 경로를 읽게되면



잘가라 CTF.. 어서와라 CVE!!

5.3 침해 결과 확인

위 취약점을 통해 시스템에서 읽을 수 없는 파일을 마음대로 읽을 수 있다는 것을 알게 됨

5.4 문제 적용 시나리오

(미안!!)

6. 대응 및 조치 방안

6.1 근본적인 보안 패치 적용

버전 업데이트:

- Jenkins Weekly 2.442 이상으로 업데이트
- Jenkins LTS 2.426.3 이상으로 업데이트
- 패치 버전에서는 expand-at 기능이 기본적으로 비활성화되도록 수정되었습니다.

6.2 주통기 기반 인프라 보안 설정

- **액세스 제어**: Jenkins 관리 콘솔에서 **'Allow CLI requests over HTTP'** 옵션을 비활성화하여 공격 접점을 최소화합니다.
- **망 분리**: Jenkins 서버를 외부 인터넷에서 직접 접근할 수 없도록 폐쇄망 또는 VPN 환경 내에 배치합니다.

6.3 임시/우회 통제 방안

- **CLI 기능 완전 차단**: 즉각적인 업데이트가 불가능한 경우, Jenkins 설정에서 CLI 접근 권한을 엄격히 제한하거나 스크립트를 통해 CLI 서블릿 접근을 차단합니다.
- **모니터링 강화**: IDS/IPS 및 WAF에서 Jenkins CLI 관련 특이 패턴(예: HTTP POST 요청 내 `@`문자 포함 인자 전송)을 탐지하도록 정책을 설정합니다.